UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 1 (10%)

Name	:	
Matric Id	:	
Section	:	

Instructions:

This assignment evaluates your ability to design and implement a structured relational database system using **SQL** using **MySQL Workbench.**

Academic Integrity

All students must work in a group of **TWO** students. Use of AI-generated SQL or code sharing is strictly prohibited. Identical or AI-generated scripts will receive **zero marks**. Provide screenshots of every instruction in all tasks.

CASE STUDY (20 MARKS)

The University Hostel Department (UHD) at Universiti Teknologi Malaysia (UTM) manages accommodation for all undergraduate students. UHD faces difficulties tracking occupancy status, rent collection, and maintenance issues across multiple hostel blocks (A, B, and C). UHD needs a database to track room types, room allocations, students, payments, and maintenance activities efficiently.

Operational background

- Each room type (Single, Double, Premium, Family) has a fixed rent, deposit, and capacity.
- Each room belongs to one type and is located on a specific floor.
- Each student is assigned one room at a time and pays rent monthly according to the room type.
- Maintenance records are logged whenever issues arise (water leak, broken furniture and other issues you may list). Each issue has a severity (LOW, MEDIUM, HIGH) and a status (OPEN, RESOLVED).
- Payments are made through methods (CASH, FPX, CARD, TNG). The finance officer tracks payment dates, methods, and remarks.

Assume the system can:

- Maintain referential integrity between all entities (room types, rooms, students, maintenance, and payments).
- Automatically reflect whether a room is occupied or vacant based on active students.
- Allow management to view monthly income reports, pending maintenance lists, and room occupancy summaries.
- Support filtered searches for conditions (For example, “All rooms with rent between RM400–RM800”).

All students are required to complete all tasks.

Task 1: Database Creation and Integrity Constraints (CLO1)

1. Create a database named `hostel_mgmt_placeyourname`, example: `hostel_mgmt_shamini`.
2. Create the following tables with entity and referential integrity (ensure these attributes exist so later tasks are possible) as shown in Table 1.

Table 1: Table Structure

Table name	Primary Key	Foreign Key	Key Attributes	Examples
room_types	type_id	-	type_name	Single, Double, Premium, Family
			rent	500.00
			deposit	300.00
			capacity	2
rooms	room_id	type_id	room_no	A101
			floor_no	1
			is_occupied	TRUE/FALSE
students	student_id	room_id	fname	Ahmad
			lname	Zaki
			status	ACTIVE/NON_ACTIVE
			checkin_date	2025-10-26
			email	ahmad_z@graduate.utm.my
maintenance	maint_id	room_id	issue_desc	Water leakage in bathroom
			severity	LOW, MEDIUM, HIGH
			status	OPEN, RESOLVED
			reported_on	2025-10-26
			resolved_on	2025-10-26
payments	payment_id	student_id	amount	500.00
			paid_on	2025-10-26
			method	CASH, FPX, CARD, TNG
			note	November Rent

3. Apply constraints: `NOT NULL, PRIMARY KEY, FOREIGN KEY, UNIQUE`.
4. Create and insert order: `room_types → rooms → students → maintenance → payments`. A room is 'occupied' if it has ≥1 student with `status='ACTIVE'`; otherwise `'VACANT'`. No orphan FKs.
5. Demonstrate schema maintenance:
    - `ALTER TABLE`: add an email column to students and mark it UNIQUE.
    - `DROP TABLE`: create a temporary test table and drop it safely.

Task 2 – Data Manipulation and Filtering (CLO2)

Each table must contain at least 10 valid records (`room_types, rooms, students, maintenance, payments`) to support all query tasks.

1. Data Insertion (DML):
Insert at least 10 rows per table with realistic and varied data as follows:

- `room_types`: Include multiple rent levels, with at least one rent value between RM400 and RM800, and ensure that deposits and capacities vary meaningfully.
- `rooms`: Distribute rooms across a minimum of three floors, ensuring that each room is correctly linked to a valid room type.

- `students`: Include both 'ACTIVE' and 'NON_ACTIVE' students. At least one student name must begin with the letter 'A', and every ACTIVE student must be assigned to an existing room.
- `maintenance`: Include diverse records with different severity levels (LOW, MEDIUM, HIGH) and statuses (OPEN, RESOLVED).
- `payments`: Include payment records covering at least two different months and using at least two different payment methods among {CASH, FPX, CARD, TNG}. Every payment must be linked to an existing student.

2. UPDATE and DELETE Operations:

- Update the `is_occupied` column in rooms to TRUE for rooms with ACTIVE students; otherwise, set it to FALSE.
- Delete maintenance records where `status = 'RESOLVED'` and the reported date is older than 60 days (if applicable).

3. Data Retrieval and Filtering Queries:
Write one query for each of the following:

- BETWEEN: Display room types where rent is between RM400 and RM800.
- LIKE: Retrieve students whose names (`fname`) start with the letter 'A'.
- IN: Display payments where the payment method is IN ('FPX', 'CARD').
- Combine AND, OR, and NOT in at least one logical filter query.

4. Functions and Expressions:
Use SQL functions as follows (no duplication of function types):

- Aggregate Function: Use at least one among COUNT(), SUM(), AVG(), MIN(), or MAX().
- String Functions: Use any two among UPPER(), LOWER(), LENGTH(), or CONCAT() in your queries.

Task 3 – Reporting and Aggregation (CLO2)

1. Create a view `v_room_status` with columns:
   (room_no, type_name, rent, floor_no, capacity, n_occupants, pending_issues, is_vacant)
   - `n_occupants` = count of students with `status='ACTIVE'` per room.
   - `pending_issues` = count of maintenance rows with status='OPEN' per room.
   - `is_vacant = TRUE` when n_occupants = 0.

2. Summary queries (for the ≥10-per-table dataset):
    - Total number of students per room type.
    - Average rent and total deposit per room type (use `ROUND()` to 2 d.p.).
    - Monthly payment totals grouped by year & month (ensure your dataset spans ≥2 months).
    - Count of `OPEN` maintenance issues per floor using `HAVING COUNT > 2`.
3. Apply SQL functions in your report queries:
    - `ROUND()`
    - `UPPER()/LOWER()`
    - `CONCAT()`
    - `CASE` to categorize room type rent as `LOW / MEDIUM / HIGH` (state your thresholds explicitly)

Submission

Each student must submit two files:

1. SQL Script File
    - File name format: `uhd_<matricno>.sql`
    - This script must contain all DDL, DML, and query statements for Task 1, Task 2, and Task 3.
    - Ensure all commands can be executed successfully without syntax or referential errors.
    - The SQL file will be used by the lecturer to re-run and validate the database creation and data manipulation steps.
    - Ensure screenshots show both the query and output window together in MySQL Workbench.
2. PDF Report
    - File name format: UHD_Hostel_<matricno>_<yourname>.pdf
    - The report must include:
        - Screenshots of each executed SQL command and its output/result.
        - Clear labelling for each screenshot (For example, Filtering Query using `BETWEEN`).
        - A short summary table listing all created tables and the number of records per table.

Important: Both files (SQL and PDF) must be submitted together.
Failure to include either file, or submission of scripts that fail to execute, will result in a mark deduction.

Submission Rules

1. The SQL script must be re-runnable from start to finish without errors.
2. Use the exact table structure and naming conventions stated in the case study.
3. Provide realistic sample data.
4. Clearly comment each section of your SQL script for clarity and readability.
5. All work must be original and individually completed. The use of AI-generated SQL scripts or sharing of code between students is strictly prohibited.
6. Late submissions will be penalized according to faculty policy.

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (`PK, FK, NOT NULL, UNIQUE`). Logical order and referential integrity preserved.	3	
	`ALTER TABLE` (add email `UNIQUE`) and `DROP TABLE` demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data `UPDATE/DELETE` executed correctly with logical results.	1	
	Accurate use of `WHERE, BETWEEN, LIKE, IN, AND/OR/NOT` queries.	3	
	Correct use of aggregate and string functions (`COUNT, CONCAT, and more)`.	2	
Task 3 Reporting & Aggregation	`VIEW` creation	2	
	Use of `GROUP BY, HAVING, ROUND, and CASE` with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
		Total (20)	